

1. To Install Linux VM, Configure GCC, Setup Git Repository, Create Project Structure, Understand Shell Architecture, Build Initial Makefile.

Task -1: Create Project Structure as follows:

```
OSSP/
|
|— .git/          # Git repository (created automatically)
|— .gitignore     # Files/folders Git should ignore
|— README.md      # Project description and instructions
|— LICENSE        # Project license (optional)
|— Makefile       # Build instructions
|
|— src/           # Source code
|   |— main.c
|   |— shell.c
|   |— parser.c
|   |— executor.c
|   └— builtin.c
|
|— include/       # Header files
|   |— shell.h
|   |— parser.h
|   |— executor.h
|   └— builtin.h
|
|— obj/           # Object files (.o) (generated)
```

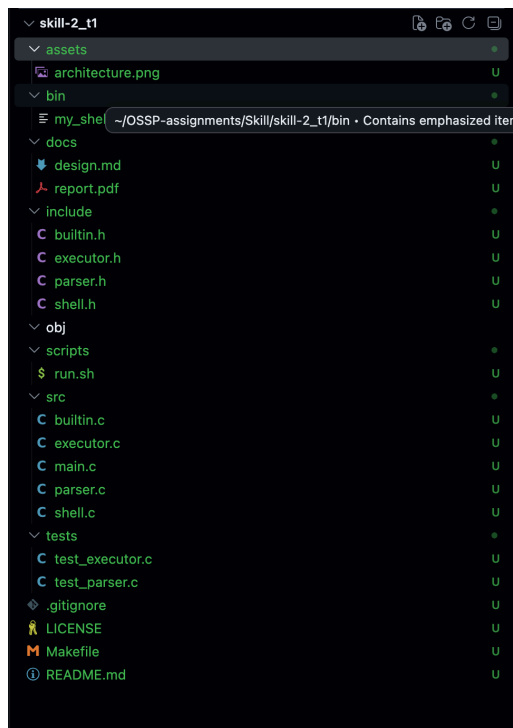
```

|
|— bin/          # Executable file (generated)
|   └─ my_shell
|
|
|— docs/         # Documentation
|   └─ design.md
|   └─ report.pdf
|
|
|— tests/        # Test programs
|   └─ test_parser.c
|   └─ test_executor.c
|
|
|— scripts/      # Helper scripts
|   └─ run.sh
|
|
└─ assets/       # Images/screenshots
    └─ architecture.png

```

Note: For creating this structure use mkdir and touch commands.

Structure:



2. To Analyze process abstraction, execute `fork()`, understand `exec()` family, analyze parent-child relationships, inspect process tree, practice system call tracing.

Task-1: Using a manual command understand the `fork()` and `exec()` system calls

Output:

```
$ pstree -p $$
bash(2041)——pstree(2200)

$ strace -f -e trace=fork,vfork,clone,execve ls
execve("/bin/ls", ["ls"], 0x7ffe...) = 0
```

Task-2: Write a C program to create a new process and accepting the user commands using `fork()` and `exec()` system calls.

Example:

```
#include <stdio.h>

#include <unistd.h>

int main() {
    pid_t pid;

    pid = fork(); // Create a new process
    if (pid < 0) {
        printf("Fork failed!\n");
    }
    else if (pid == 0) {
```

```

        // Child Process
        printf("This is the Child Process.\n");
        printf("Child PID = %d\n", getpid());
    }
    else {
        // Parent Process
        printf("This is the Parent Process.\n");
        printf("Parent PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);
    }
    return 0;
}

```

Task2 code:

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_LINE 256
#define MAX_ARGS 32

int main() {
    char line[MAX_LINE];
    char *args[MAX_ARGS];
    pid_t pid;

```

```

while (1) {
    printf("myshell> ");
    fflush(stdout);

    if (fgets(line, sizeof(line), stdin) == NULL)
        break;                // Ctrl+D (EOF)

    line[strcspn(line, "\n")] = '\0';    // strip newline

    if (strlen(line) == 0)
        continue;              // empty command
    if (strcmp(line, "exit") == 0)
        break;

    // Tokenize into argv[]
    int argc = 0;
    char *tok = strtok(line, " ");
    while (tok != NULL && argc < MAX_ARGS - 1) {
        args[argc++] = tok;
        tok = strtok(NULL, " ");
    }
    args[argc] = NULL;

    pid = fork();              // Create a new process

    if (pid < 0) {
        printf("Fork failed!\n");
    }
}

```

```

    }

    else if (pid == 0) {
        // Child Process

        printf("This is the Child Process.\n");
        printf("Child PID = %d\n", getpid());


        execvp(args[0], args);          // Replace child image with the command

        // Only reached if execvp() fails
        printf("Command not found: %s\n", args[0]);
        exit(1);
    }

    else {
        // Parent Process

        printf("This is the Parent Process.\n");
        printf("Parent PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);


        int status;

        waitpid(pid, &status, 0);        // Wait for the child to finish

        if (WIFEXITED(status))

            printf("Child exited with status %d\n\n", WEXITSTATUS(status));

    }
}

return 0;
}

```

Output:

```
myshell> ls
This is the Parent Process.
Parent PID = 3050
Child PID = 3051
This is the Child Process.
Child PID = 3051
file1.txt  file2.txt  Makefile
Child exited with status 0
```

```
myshell> pwd
This is the Parent Process.
Parent PID = 3050
Child PID = 3052
This is the Child Process.
Child PID = 3052
/home/user/OSSP
Child exited with status 0
```